



Universidade do Minho
Licenciatura em Ciências da Computação

Unidade Curricular de Computação Gráfica

Ano Letivo de 2023/2024

Transformações Geométricas – Fase 2

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Abril, 2024

CG

Data de Recepção	
Responsável	
Avaliação	
Observações	

Transformações Geométricas – Fase 2

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Abril, 2024

Introdução

O presente relatório é referente à segunda fase de um projeto desenvolvido no âmbito da unidade curricular de Computação Gráfica, que se centra no desenvolvimento de um cenário gráfico 3D representativo do sistema solar através da leitura e interpretação de um ficheiro XML e do uso da primitiva gráfica (esfera).

Esta fase requer o desenvolvimento de cenas organizadas hierarquicamente utilizando transformações geométricas. Tendo esse propósito em mente, foram acrescentadas algumas estruturas de dados e atualizadas as já existentes.

Ao longo deste relatório irão ser explicadas, de forma detalhada, todas as decisões tomadas.

Índice

Introdução	i
Índice	ii
Índice de Figuras	iii
1. Novas Estruturas	1
1.1. <i>Transform</i>	1
1.2. <i>Group</i>	1
1.3. <i>Config</i>	2
2. Atualização de funções existentes	3
2.1. <i>xmlToConfig</i>	3
2.2. <i>drawModels</i>	4
3. Anel de Saturno	5
4. Câmara FPS	7
5. Sistema Solar	8
6. Resultados	9
6.1. Capturas referentes aos testes fornecidos	9
6.2. Captura do Anel de Saturno	10
6.3. Capturas do Sistema Solar implementado	11
7. Conclusão	13

Índice de Figuras

Figura 1 – Geração dos pontos dos vértices para o anel	5
Figura 2 – test_2_1	9
Figura 3 – test_2_2	9
Figura 4 – test_2_3	9
Figura 5 – test_2_4	9
Figura 6 – Anel de Saturno	10
Figura 7 – Modelo estático do Sistema Solar	11
Figura 8 – Saturno	11
Figura 9 – Terra e Lua	11
Figura 10 – Modelo estático do Sistema Solar com rotações	12
Figura 11 – Modelo estático do Sistema Solar com rotações	12

1. Novas Estruturas

1.1. *Transform*

Foi criada a estrutura *Transform* de acordo com o excerto de código presente abaixo.

```
struct Transform
{
    int type;
    float angle;
    Point coords;
};
```

Esta estrutura contém os campos necessários para guardar as informações de qualquer uma das transformações geométricas:

- *type* permite guardar o tipo de transformação. Aceita os valores “0”, “1” e “2” que correspondem, respetivamente, a translação, rotação e escala;
- *angle* contém o ângulo de rotação. No caso em que a transformação é uma translação ou uma escala, *angle* assume o valor 0;
- *coords* contém os parâmetros “x”, “y” e “z” de uma transformação, Este campo é comum a todas as transformações.

1.2. *Group*

De forma a ser mantida uma hierarquia e tendo em conta a linguagem utilizada nos ficheiros XML, foi implementada a estrutura *Group*, constituída por três vetores:

- vetor de transformações;
- vetor com o nome dos modelos a desenhar;
- vetor de grupos.

```
struct Group
{
    vector<Transform> transforms;
    vector<string> models;
    vector<Group> groups;
};
```

Esta estrutura de dados permite guardar as transformações geométricas e os modelos que dizem respeito a cada grupo. De forma a haver uma hierarquia, é possível ter grupos que contêm grupos. Assim, um grupo “filho” herda as transformações geométricas que afetaram o grupo “pai”.

É também importante referir que na estrutura *Group* só é guardado, no vetor *models*, o nome da primitiva a desenhar.

1.3. Config

Os pontos necessários para desenhar cada um dos *models* será guardado no campo *models* da estrutura *Config*. Desta forma evita-se guardar repetidamente informações de uma mesma primitiva geométrica. Sendo o campo *models* um *map*, o acesso aos pontos da primitiva é imediato.

```
struct Config
{
    int window[2];
    float position[3];
    float lookAt[3];
    float up[3];
    float projection[3];
    map<string, vector<Point>> models;
    Group group;
};
```

2. Atualização de funções existentes

2.1. *xmlToConfig*

Na primeira fase era apenas necessário recolher informação de um elemento *group* do ficheiro XML. Havendo agora a possibilidade de existir vários grupos foi então necessário atualizar a função *xmlToConfig*. Desta forma, foi criada a função *addGroup*.

addGroup, além de permitir guardar as informações que dizem respeito às transformações geométricas, permite também adicionar recursivamente grupos “filho” a grupos “pai”.

```
Group addGroup (XMLElement *element, Config *conf);
```

Importa que seja mencionado o facto de que ao contrário de outros elementos que são lidos do ficheiro XML, quando é feito *parse* de um conjunto de transformações (*tag transform*), é necessário respeitar a ordem com que surgem as transformações.

```
XMLElement *pTransform = group_pointer->FirstChildElement("transform");
if (pTransform != nullptr)
{
    XMLElement *pCurrentTransform = pTransform->FirstChildElement();

    while (pCurrentTransform != nullptr)
    {
        .
        .
        .
        pCurrentTransform = pCurrentTransform->NextSiblingElement();
    }
}
```

É ainda relevante ter em conta que, sempre que é guardado um nome de uma primitiva numa estrutura *Group*, este tem também de ser adicionado à estrutura *Config*.

No entanto, enquanto é oportuno guardar todas as vezes que surge o nome de uma primitiva no campo *models* da estrutura *Group* (uma vez que este campo contém informações sobre quais e quantos elementos vão ser desenhados), na estrutura *Config* apenas interessa guardar quais os diferentes modelos a desenhar. Este comportamento é salvaguardado pelas características do campo *models* da estrutura *Config*.

2.2. *drawModels*

Foi necessário, igualmente, atualizar a função *drawModels* que é responsável por desenhar as primitivas gráficas.

Nesta fase pretende-se desenhar os modelos de acordo com uma hierarquia. Tendo este facto em conta, fez-se uso das funções *glPushMatrix* e *glPopMatrix*. Estas funções permitem gravar e restaurar o estado da matriz, sendo assim útil para acumular transformações e, quando necessário, ser possível voltar a um estado da matriz anterior. Para percorrer e desenhar as primitivas dos grupos “filho”, foi utilizada a função *drawModels* recursivamente.

```
void drawModels(Config *conf, Group group)
{
    glPushMatrix();
    .
    .
    .
    vector<Group> groups = group.groups;
    if (!groups.empty())
    {
        for (vector<Group>::iterator it = groups.begin(); it !=
groups.end(); it++)
        {
            drawModels(conf, *it);
        }
    }

    glPopMatrix();
}
```

3. Anel de Saturno

Com o intuito de representar o anel de Saturno, foi tomada a decisão de adicionar à aplicação **generator** a primitiva gráfica *ring*.

A função que gera os pontos dos vértices necessários para o desenho desta primitiva são:

- *innerRadius* – raio do círculo interior;
- *outerRadius* – raio do círculo exterior;
- *slices* – número de fatia/divisões que o anel terá;
- *filename* – nome do ficheiro onde vai ser guardada a informação.

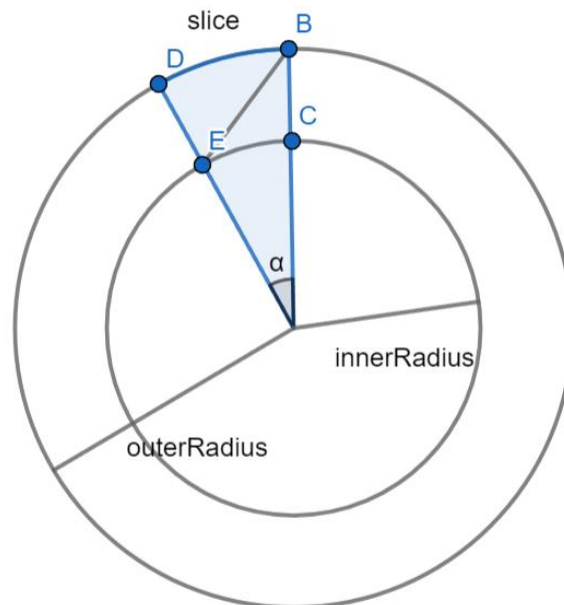


Figura 1 - Geração dos pontos dos vértices para o anel

Inicialmente é calculado o valor do ângulo *alphaInc*, valor esse necessário para o cálculo dos vértices dos triângulos do *ring*.

$$\alpha Inc = \frac{2\pi}{slices}$$

De seguida, recorrendo a coordenadas polares e cartesianas, são calculados os pontos B, C, D e E.

Tendo em conta a necessidade de que o anel seja visto tanto quando a câmara está posicionada acima ou abaixo do *ring*, são guardados os pontos que permitem desenhar quatro triângulos, dois deles com a face voltada para cima e outros dois com a face voltada para baixo.

Este processo é repetido para cada *slice*, incrementando o valor de *alphaInc* a cada iteração.

4. Câmara FPS

De maneira a ser possível explorar melhor a cena do Sistema Solar foi implementada uma câmara FPS.

O utilizador pode alternar entre a câmara FPS e o modo explorador.

Esta nova câmara permite deslocar pelo mundo usando "W", "S", "A" e "D" e permite olhar em volta usando as setas do teclado.

5. Sistema Solar

A procura de medidas para desenhar um sistema solar à escala não foi fácil de concretizar, tendo sempre em conta a necessidade de ser desenhado um sistema solar realista.

Devido à extremamente grande dimensão do Sol relativamente aos planetas, foi tomada a decisão de reajustar a sua dimensão.

Cumprindo a noção de hierarquia, dentro de um *group* foram implementados outros *groups* respetivos a cada um dos astros, cada um com as suas translações e escalas relativas ao astro em questão.

No caso específico da Terra, dentro do seu *group* implementou-se ainda um outro *group* que diz respeito à Lua.

Para o segundo teste do Sistema Solar, recorreu-se à representação em XML de funções do *glut*, tais como *rotate*, *scale* e *translate* a fim de dar um efeito mais realista ao Sistema Solar.

6. Resultados

6.1. Capturas referentes aos testes fornecidos

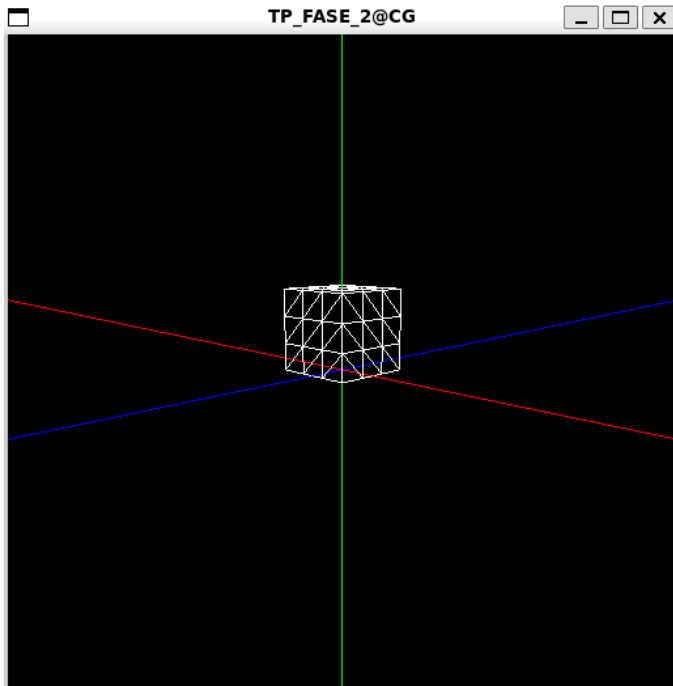


Figura 2 - test_2_1

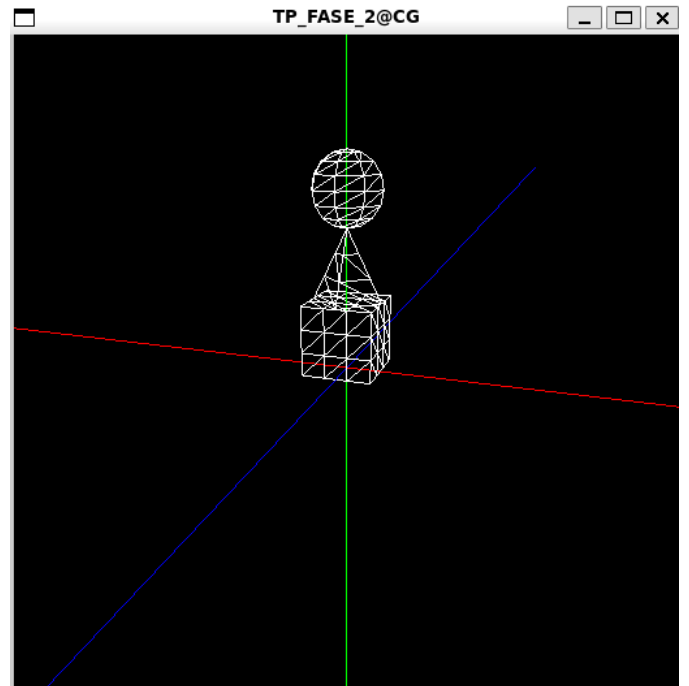


Figura 3 - test_2_2

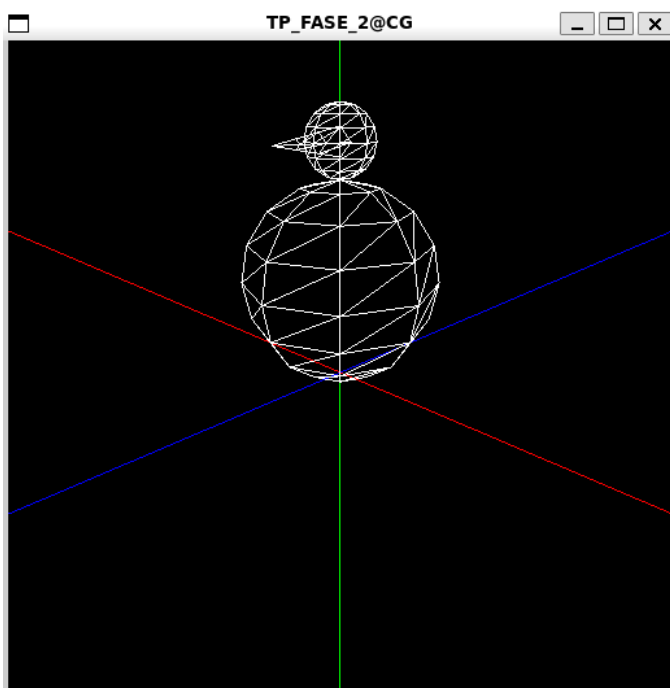


Figura 4 - test_2_3

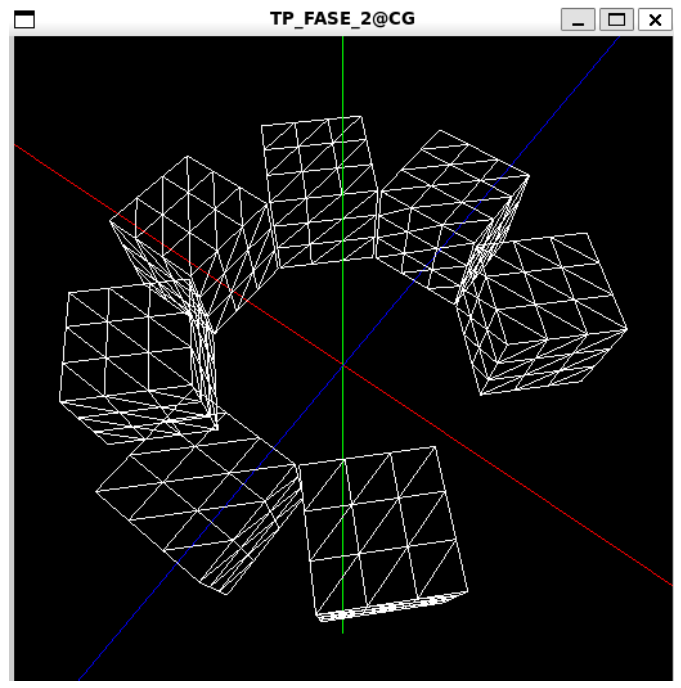


Figura 5 - test_2_4

6.2. Captura do Anel de Saturno

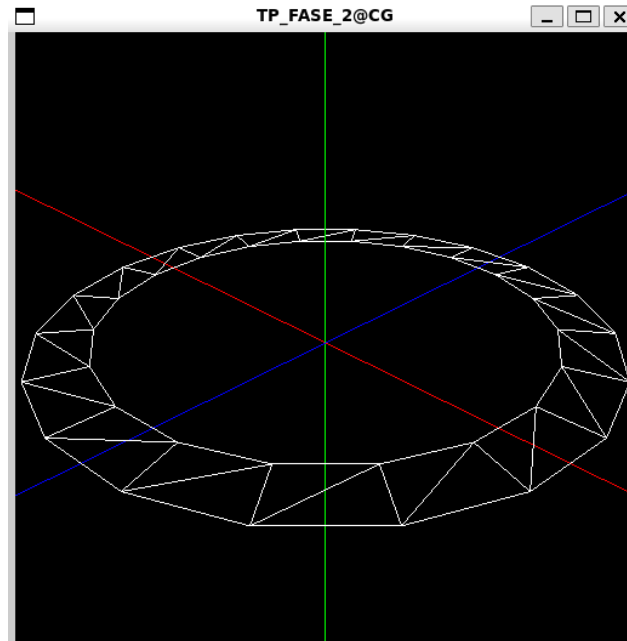


Figura 6 - Anel de Saturno

6.3. Capturas do Sistema Solar implementado

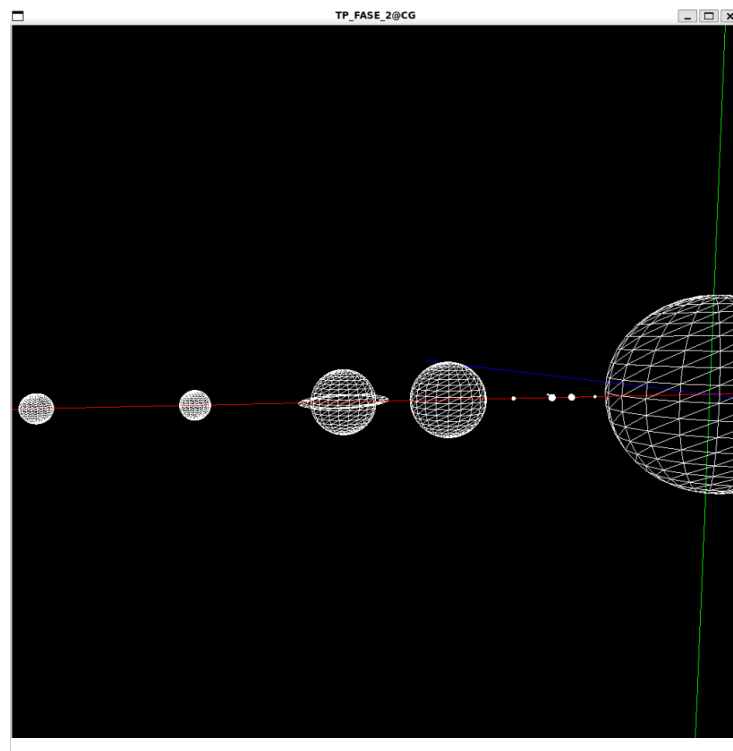


Figura 7 - Modelo estático do Sistema Solar

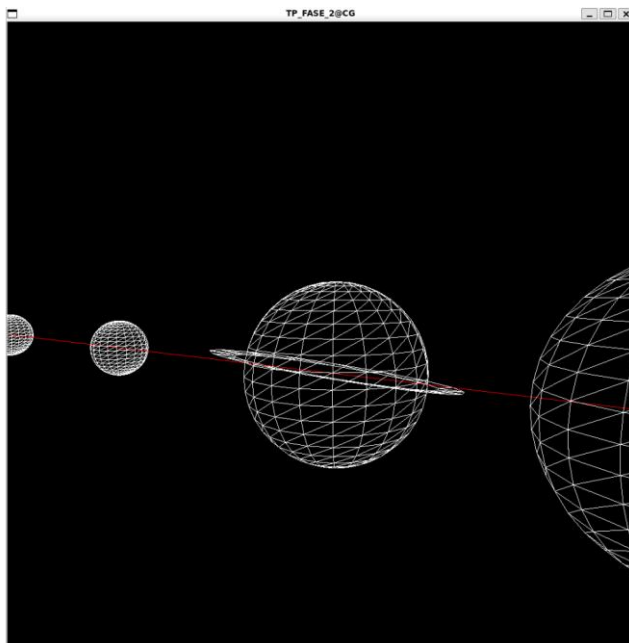


Figura 8 - Saturno

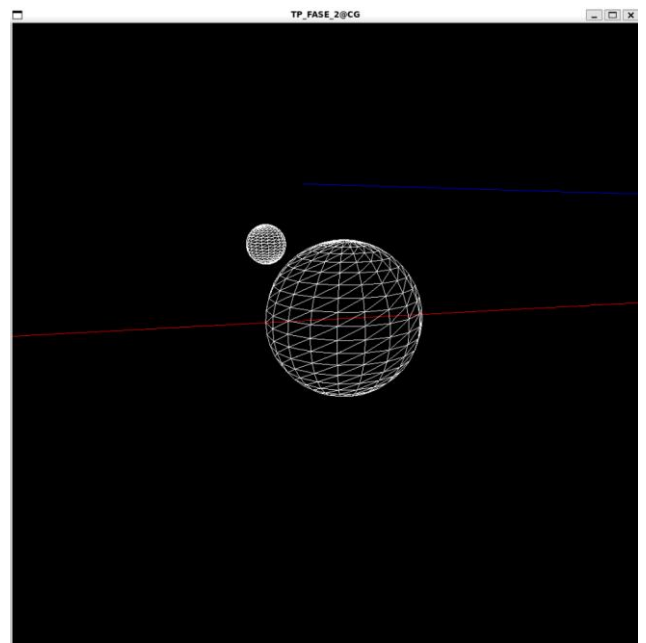


Figura 9 - Terra e a Lua

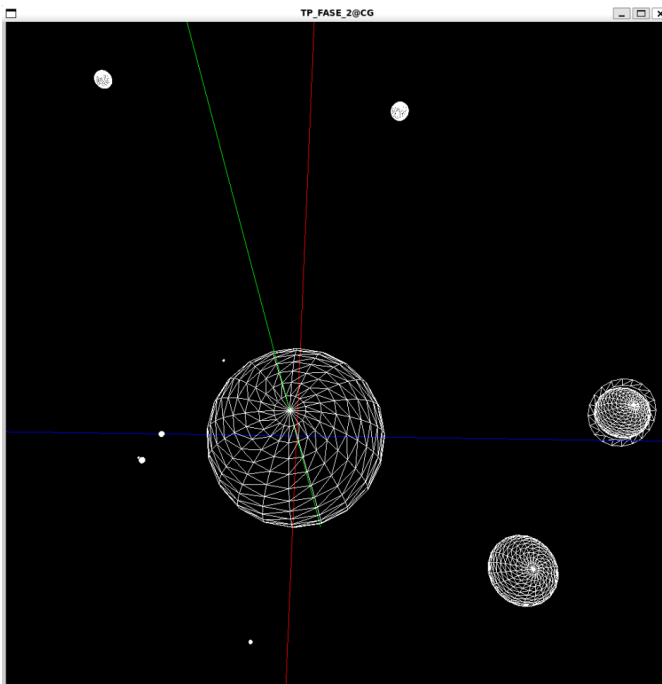


Figura 10 - Modelo estático do Sistema Solar com rotações

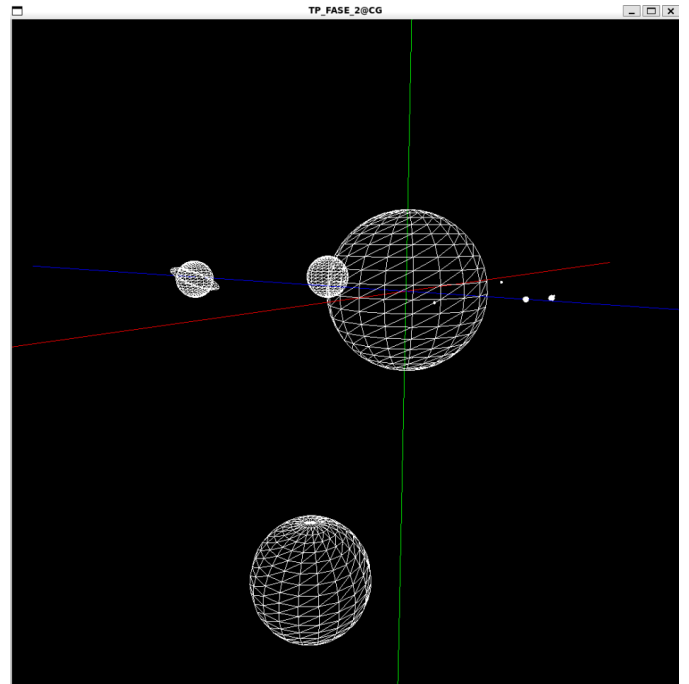


Figura 11 - Modelo estático do Sistema Solar com rotações

7. Conclusão

Esta fase do projeto concentrou-se na organização hierárquica das cenas 3D, neste caso em específico, os astros que formam o Sistema Solar, utilizando para tal transformações geométricas e estruturas de dados adequadas.

Foram introduzidas novas estruturas e atualizadas funções já existentes para lidar com grupos e transformações de forma eficiente.

A implementação do anel de Saturno e da câmara FPS teve como por objetivo enriquecer a representação visual do Sistema Solar.

Em suma, considera-se o balanço do trabalho realizado positivo dado que as dificuldades sentidas foram superadas e cumpridos todos os requisitos.